

fix-hostfiles

Unblocking a domain without breaking /etc/hosts, and how it is built

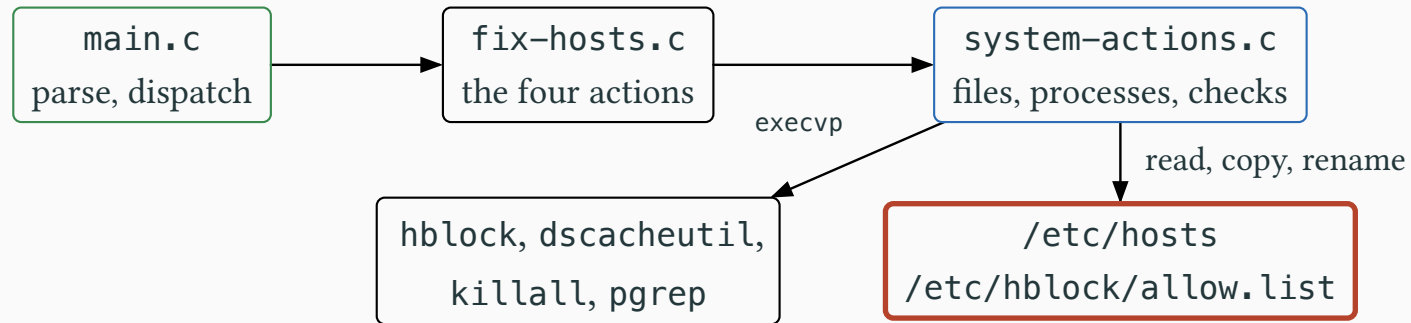
October 2026

What fix-hostfiles is

- hblock maps ad, tracking and malware domains to `0.0.0.0` in `/etc/hosts`
- Sometimes it blocks a domain you need; fix-hostfiles is the way back
- **Four actions**, one at a time: `prep`, `restore`, `--add`, `--flush`
- **No shell**: every program it runs is started with `fork` and `execvp`
- **Atomic**: the hosts file is replaced with one `rename(2)`, never rewritten in place

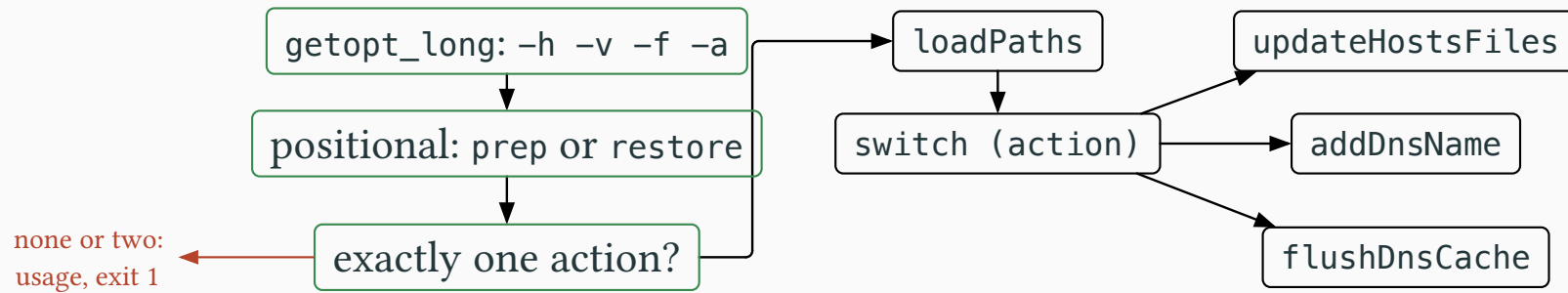
About 1,100 lines of C11. The interesting part is the few dozen that touch `/etc/hosts`.

Architecture



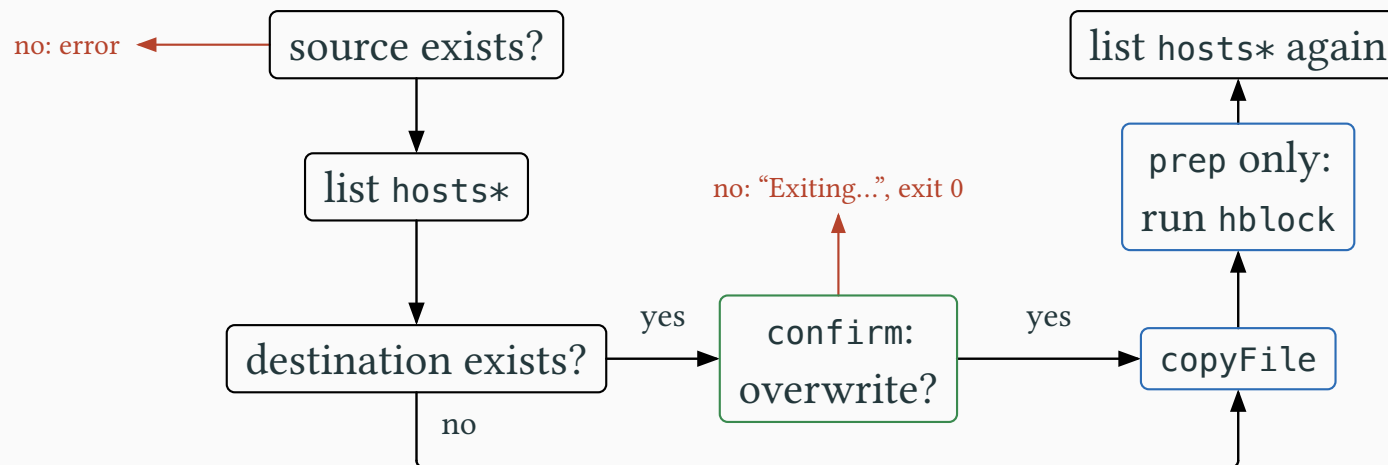
`main.c` never touches a file; `fix-hosts.c` decides, `system-actions.c` does.

The command line: `main()`



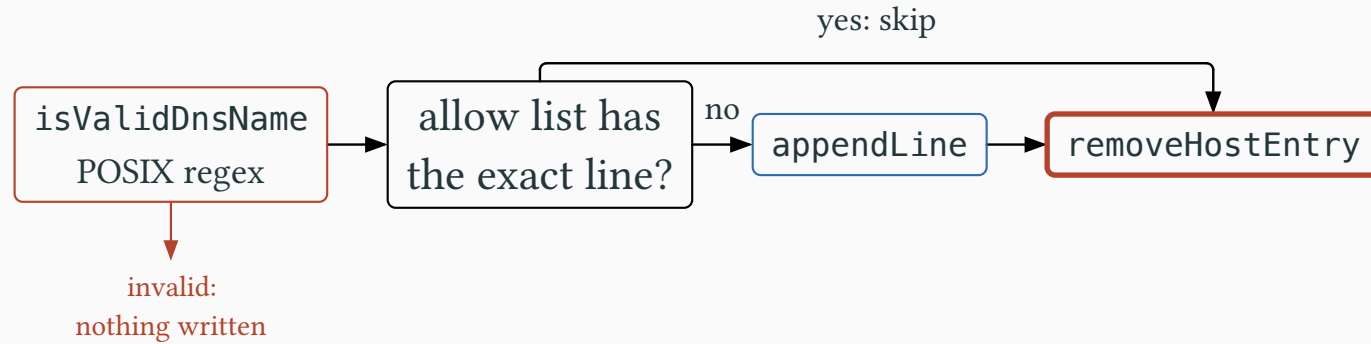
- `selectAction` refuses a second action, so `--add x --flush` fails before anything runs
- `FIX_HOSTFILES_ETC_DIR` and `..._HBLOCK_DIR` move every path into a test fixture

prep and restore: one function, two directions



Action	Source	Destination
prep	/etc/hosts	/etc/hosts-ORIG
restore	/etc/hosts-ORIG	/etc/hosts

--add: unblocking one domain



- Adding twice leaves the allow list as adding once
- A hosts line goes only if a whitespace-separated token equals the name exactly: `0.0.0.0 ads.example.com` goes, `0.0.0.0 myads.example.com` stays
- Comments (`# ...`) are never matched

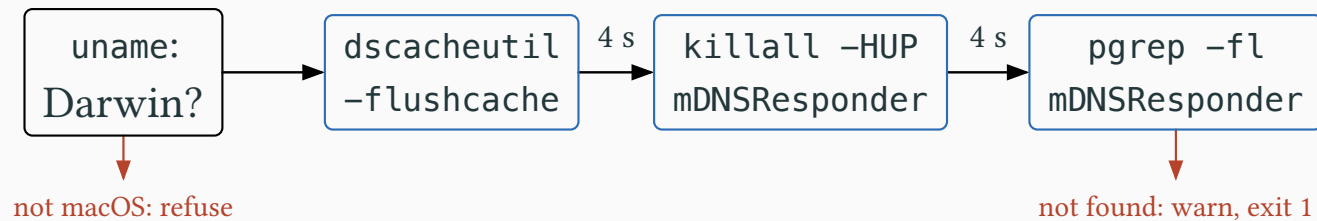
Atomic replacement: removeHostEntry



Any failure before rename: unlink the temporary file; /etc/hosts is untouched.

- The temporary file sits beside the hosts file, so rename stays on one filesystem
- A reader sees the whole old file or the whole new one, never half
- Owner and mode are copied from the original before the swap

`--flush`: three programs, no shell



`runCommand` forks, the child calls `execvp(argv[0], argv)`, the parent `waitpids` and retries on `EINTR`. Arguments are an array, so a domain name can never become shell syntax.

The laws it keeps

Write H for a hosts file, A for the allow list, d, e for DNS names.

Removal idempotence

$$\text{remove}_d(\text{remove}_d(H)) = \text{remove}_d(H)$$

Distinct removals commute

$$\text{remove}_d \circ \text{remove}_e = \text{remove}_e \circ \text{remove}_d$$

Insertion idempotence

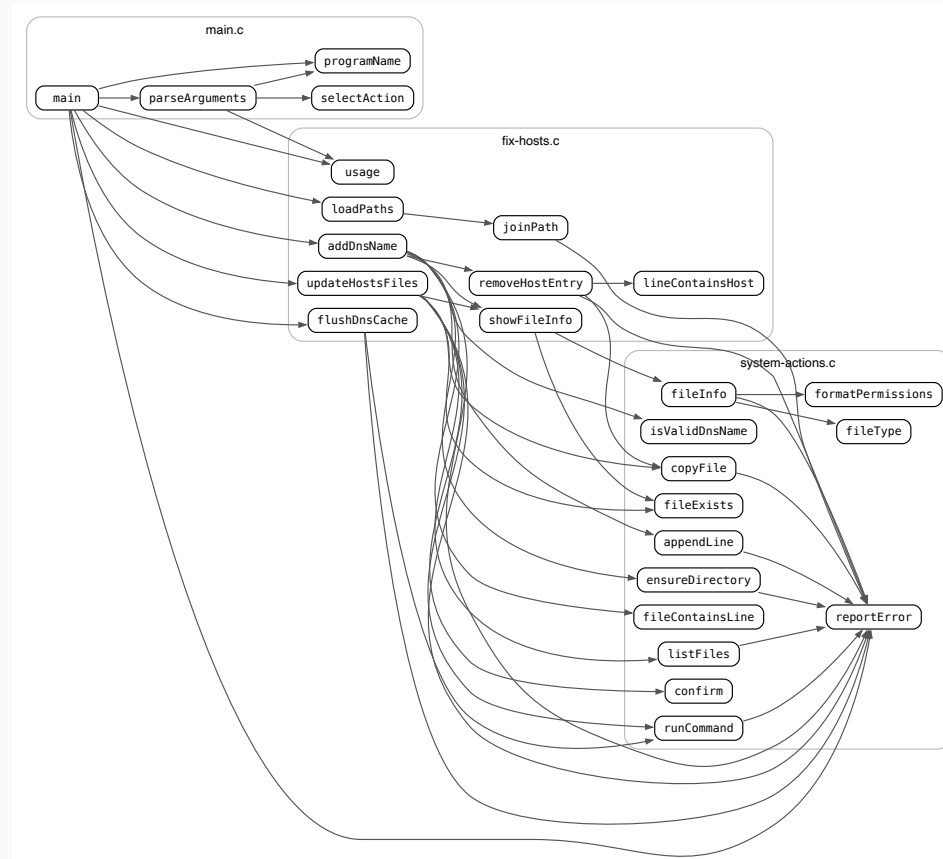
$$\text{insert}_d(\text{insert}_d(A)) = \text{insert}_d(A)$$

Prep-restore round trip

$$(\text{restore} \circ \text{prep})(H) = H$$

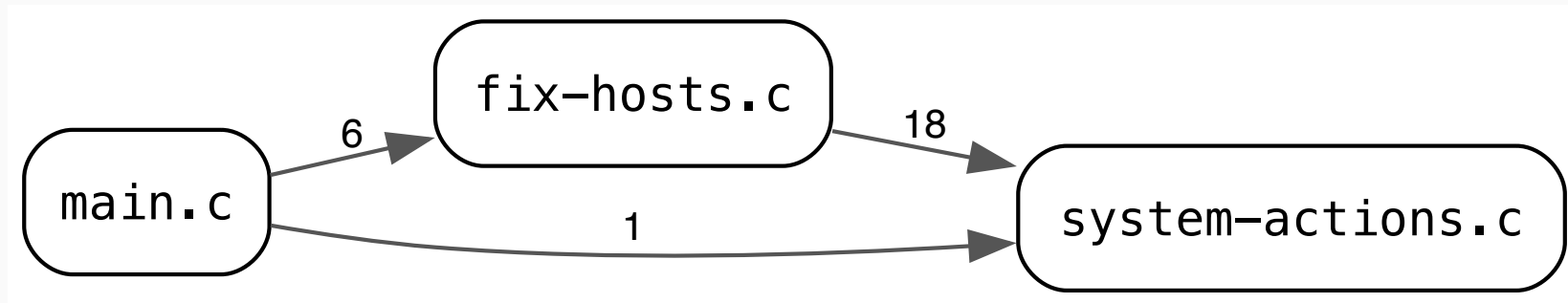
Twelve laws in all, from `algebra/fix-hostfiles-algebra.tex`. The tests check examples; the spec proposes property-based tests for these. The same formulas in LaTeX and in Typst differ mainly in `\Remove{d}` versus `"remove"_d`.

Every call



Every call between fix-hostfiles' own functions: 26 functions, 42 calls. Generated from clang's AST, not drawn.

Calls between files



The same graph folded by file; numbers are distinct caller-callee pairs. Nothing calls back into `main.c`, and `system-actions.c` calls nothing above it.

Where the functions are

File	Functions	Role
src/system-actions.c	13	files, processes, validation, error reports
src/fix-hosts.c	9	paths and the four actions
src/main.c	4	argument parsing and dispatch

Read from `functions.csv` when the slides compile: Typst reads data itself; the fsb Beamer slides had these numbers typed in.

- About 1,100 lines of C11
- Nine end-to-end tests in Bash, run in a temporary fixture
- Built with `-Wall -Wextra -Wpedantic -Werror`
- Started 25 February 2024
- Algebraic spec in LaTeX: twelve laws
- API docs with Doxygen, a man page

How these slides were made

1. **Call graph** from clang's own parse:

```
clang -fsyntax-only -Xclang -ast-dump=json src/main.c
```

2. **Filter**: a short Python script keeps calls between the program's own functions and writes Graphviz files and a CSV; dot draws SVGs
3. **Diagrams**: drawn by hand with the `fletcher` package, checked against the graph
4. **Slides**: Typst with `touying`, theme *metropolis* (moloch's ancestor)

`make` in `docs/talk/` rebuilds everything from the current code; `make watch` recompiles on every save in well under a second.